

Solving JSON Quote Escaping with Custom Delimiters

Using triple backticks (```) as a direct replacement for double quotes in problematic JSON fields is an excellent, highly practical developer hack. Because LLMs are extensively trained on Markdown, they are incredibly good at writing raw code inside backticks without attempting to escape internal quotes.

By allowing the LLM to output a "Quasi-JSON" structure, we offload the escaping work to a simple regex pre-processor on our system before passing the payload to a standard JSON parser.

1. The Target Output Structure

Instead of forcing the LLM to escape quotes, we instruct it to output the code value using raw triple backticks *instead* of double quotes:

```
{
  "id": "task_01",
  "language": "python",
  "code": ```def greet(name):
    print(f"Hello, {name}!")
    return 'Done'```
  "status": "active"
}
```

Note: The code above is technically invalid standard JSON, but it is incredibly easy for us to sanitize.

2. The System Prompt

To get the LLM to reliably output this format, use a prompt that explicitly forbids quote escaping inside the designated field:

You must output your response as a JSON object.

CRITICAL INSTRUCTION FOR THE "code" FIELD:

Do NOT wrap the "code" field value in double quotes. Instead, wrap it in triple backticks (```).

Inside the triple backticks, write the code exactly as-is. Do NOT escape any double quotes ("), single quotes ('), or newlines (\n).

Example format:

```
{
  "id": "123",
  "code": ```your raw "un-escaped" code here```
}
```

3. Pre-processor Implementation

To make this work in your application, run the raw LLM string through a pre-processor before calling your JSON parser. The pre-processor locates the triple backticks, extracts the raw string, properly escapes it using your language's standard library, wraps it in standard double quotes, and replaces the original block.

Python Implementation

```
import re
import json

def parse_quasi_json(raw_llm_output: str) -> dict:
    """
    Converts a Quasi-JSON string containing unescaped triple-backtick
    fields into standard JSON, then parses it.
    """
    def sanitize_block(match):
        raw_content = match.group(1)
        # json.dumps automatically handles escaping quotes,
        backslashes, and newlines
        return json.dumps(raw_content)

    # Regex looks for: ``` followed by any characters (including
    newlines) up to the closing ```
    # It uses a non-greedy match (.*?) to handle multiple code fields
    if present.
    clean_json_str = re.sub(r'```(.*?)```', sanitize_block,
raw_llm_output, flags=re.DOTALL)

    return json.loads(clean_json_str)

# --- Verification ---
raw_input = """
{
  "id": "task_01",
  "language": "python",
  "code": ```def greet(name):
    print(f"Hello, {name}!")
    return 'Done'```
  "status": "active"
}
"""

parsed_data = parse_quasi_json(raw_input)
```

```
print("Successfully parsed!")
print(parsed_data["code"])
```

JavaScript / Node.js Implementation

```
/**
 * Converts a Quasi-JSON string containing unescaped triple-backtick
 * fields into standard JSON, then parses it.
 */
function parseQuasiJSON(rawLLMOutput) {
  const cleanJSONStr = rawLLMOutput.replace(/```([\s\S]*?)```/g,
    (match, rawContent) => {
      // JSON.stringify handles escaping quotes, backslashes, and
      // newlines
      return JSON.stringify(rawContent);
    });

  return JSON.parse(cleanJSONStr);
}

// --- Verification ---
const rawInput = `{
  "id": "task_01",
  "language": "python",
  "code": "\\`\\`\\`def greet(name):
    print(f'Hello, {name}!')
    return 'Done'\\`\\`\\`,
  "status": "active"
}`;

const parsedData = parseQuasiJSON(rawInput);
console.log("Successfully parsed!");
console.log(parsedData.code);
```

4. Why This Works Perfectly

1. **LLM Friendliness:** LLMs naturally write markdown blocks. Forcing them to escape quotes inside standard JSON strings goes against their pre-training, whereas writing raw code inside backticks aligns perfectly with it.
2. **Deterministic Sanitization:** Computers are great at escaping strings; LLMs are not. By offloading the string escaping to `json.dumps()` or `JSON.stringify()`, you guarantee 100% syntactical validity every time.
3. **No String Collisions:** If your code actually contains triple backticks (which is rare), you can easily modify the delimiter to something even more obscure, like `|||CODE|||`, and adjust the regex accordingly.

The Gold Standard: Structured Outputs (JSON Schema)

If you are interacting with these models via an API (OpenAI, Anthropic, Google Gemini, etc.), you shouldn't rely on raw prompting for JSON anymore. You should use Structured Outputs (or `response_format` with a JSON Schema).

When you enforce a strict JSON schema at the API level, the provider forces the model's token selection to only choose valid JSON tokens. It makes it mathematically impossible for the model to forget an escape quote.

Not relevant for me though because I am interacting with LLM from my agent via WebChat interface (because of license restrictions)